

Roomate

Sistema inteligente para la gestión de convivencia,
gastos compartidos y organización del hogar



José Manuel Ortega Soto



Tutor: Álvaro Osorio



Centro: VictoriaFP



Titulación: C.F.G.S. DAM



Convocatoria: Junio 2026



Flutter

iOS · Android · Web



Supabase

PostgreSQL · RLS



GPT-4o

IA Contextual



Stripe

Pagos PCI



Firebase FCM

Notificaciones



C.F.G.S. Desarrollo de
Aplicaciones Multiplataforma



Curso
2025 - 2026

Índice de contenidos

Roomate · C.F.G.S. Desarrollo de Aplicaciones Multiplataforma · 2025–2026

01	Memoria del Proyecto Requisitos · Planificación · Stack	02	Aplicación Final Capturas · Flujos · Accesibilidad
03	Documentación Técnica Arquitectura · DB · RLS · APIs	04	Manual de Usuario Instalación · Módulos · FAQ
05	Presentación Final Slides · Demo · Defensa	06	Diagrama de Gantt y Cronograma Cronograma · Hitos · Sprints
07	Evaluación del Proyecto Competencias · Conclusiones		

1. Memoria del Proyecto	5
1.1 Introducción	5
1.2 Objetivos y propuesta de valor	5
1.3 Tecnologías utilizadas	5
1.4 Planificación y sprints	6
1.5 Requisitos funcionales y no funcionales	8
2. Aplicación Final	9
2.1 Capturas relevantes	9
2.2 Funcionalidades principales	9
2.3 Flujo de navegación	9
2.4 Código fuente y accesibilidad	10
3. Documentación Técnica	11
3.1 Instalación y puesta en marcha	11
3.2 Arquitectura del sistema (Frontend y Backend)	11
3.3 Base de datos (18 tablas PostgreSQL)	12
3.4 Seguridad — Políticas RLS (64 reglas)	12
3.5 APIs y Edge Functions	12
3.6 Diagrama GRAM y estrategia de testing	13

4. Manual de Usuario

14

4.1 Instalación e inicio de sesión

14

4.2 Uso de la aplicación

14

4.3 Preguntas frecuentes

15

4.4 Configuración y privacidad

15

5. Presentación Final

17

5.1 Estructura de la defensa

17

5.2 Demo y puntos de apoyo oral

17

6. Diagrama de Gantt y Cronograma

18

6.1 Cronograma y timeline

18

6.2 Fases, hitos y sprints

18

7. Evaluación del Proyecto

19

7.1 Resultados obtenidos

19

7.2 Incidencias y soluciones

19

7.3 Mejoras futuras

20

7.4 Conclusiones

20

1. Memoria del Proyecto

20/20

Requisitos funcionales

Cobertura 100%

3

Plataformas

iOS · Android · Web

5

Integraciones

OpenAI · Stripe · FCM · Resend · ML Kit

<20 EUR

Coste infra

Arquitectura serverless

Introduccion y problema detectado

Roomate nace de una necesidad cotidiana: la gestion de hogares compartidos se apoya en herramientas dispersas, mensajes informales y acuerdos sin trazabilidad. El resultado es friccion economica, organizativa y comunicativa entre convivientes.

Dimension	Problema detectado	Impacto
Economica	Gastos comunes repartidos en notas o chats.	Errores, deudas y conflictos.
Organizativa	Tareas y rutinas sin seguimiento formal.	Sensacion de reparto injusto.
Comunicativa	Informacion importante perdida en mensajeria.	Falta de trazabilidad.

Propuesta de valor y publico objetivo

- Objetivo funcional**
Centralizar la operativa del hogar y eliminar friccion entre convivientes.
- Objetivo tecnico**
App multiplataforma segura con backend serverless y observabilidad suficiente.
- Alcance**
iOS, Android y Web desde una unica base de codigo con finanzas, organizacion, chat e IA.
- Publico objetivo**
Estudiantes, jovenes profesionales y grupos que comparten vivienda y gastos.

Stack tecnologico justificado

Capa	Tecnologia	Motivo de eleccion
Cliente	Flutter 3 / Dart	Un codigo, tres plataformas. UI reactiva y consistente.
Estado	Riverpod 2	Providers tipados, asincronia clara, invalidacion selectiva.
Navegacion	GoRouter	Enrutado declarativo con redirecciones por auth.
Modelos	Freezed	Inmutabilidad, serializacion y seguridad de tipos.

Capa	Tecnologia	Motivo de eleccion
Backend	Supabase	PostgreSQL, Auth, Storage, Realtime y Edge Functions.
IA	OpenAI GPT-4o	Chat contextual, analisis multimodal y OCR asistido.
Pagos	Stripe	PCI-compliant, SDK Flutter nativo, modo test.
Push	Firebase FCM	Notificaciones iOS/Android. Plan gratuito Spark.

Arquitectura por capas

Presentacion Pantallas, widgets y navegacion por features.	Estado Riverpod, AsyncNotifier e invalidacion reactiva.	Repositorios Acceso a datos, RPC y normalizacion de dominio.	Backend Supabase, auth, storage, funciones y secretos.
--	---	--	--

La app sigue arquitectura por capas orientada a features. La presentacion delega en providers; estos encapsulan estado y mutaciones; los repositorios abstraen Supabase y servicios externos; la seguridad critica reside en DB y Edge Functions. Este desacoplamiento permite evolucionar interfaz sin tocar SQL y reforzar seguridad sin depender del cliente.

Planificacion — Sprints y cronograma

Sprint	Periodo	Objetivo	Entregable
S1	03/02 - 16/02	Bootstrap	Flutter + Supabase + Auth funcional
S2	17/02 - 02/03	Core DB	17 tablas + RLS + CRUD gastos
S3	03/03 - 16/03	Finanzas	Suscripciones + presupuestos + deudas
S4	17/03 - 30/03	Colaboracion	Tareas + compra RT + chat
S5	31/03 - 13/04	Integraciones	Stripe + FCM + Edge Functions
S6	14/04 - 27/04	IA + Polish	GPT-4o + OCR + dark mode
--	28/04 - 11/05	Bug fixing	Pruebas con usuarios + feedback
--	12/05 - 28/05	Documentacion	4 docs PDF + presentacion final



Requisitos no funcionales (RNF)

ID	Requisito no funcional	Categoria	Cumplimiento
RNF01	Tiempo de respuesta de pantalla < 3 segundos en conexion 4G.	Rendimiento	Validado con usuarios.
RNF02	Sincronizacion realtime < 1 segundo para actualizaciones del hogar.	Rendimiento	Supabase Realtime.
RNF03	La app debe funcionar en iOS 16+, Android 10+ y navegadores modernos.	Compatibilidad	Testeado en 3 plataformas.
RNF04	Ningun usuario puede leer datos de otro hogar (aislamiento multi-tenant).	Seguridad	64 politicas RLS.
RNF05	Credenciales y secretos nunca expuestos en el cliente Flutter.	Seguridad	Edge Functions + .env.
RNF06	Los pagos cumplen PCI DSS (delegado a Stripe).	Seguridad	Stripe PCI L1.
RNF07	Interfaz en espanol. Soporte de tema claro y oscuro.	Usabilidad	Material 3 theming.
RNF08	Codigo fuente documentado y estructurado por features.	Mantenibilidad	Arquitectura por capas.
RNF09	Coste de infraestructura < 20 EUR/mes para el piloto.	Coste	Planes gratuitos verificados.
RNF10	El sistema debe degradar funcionalidad no critica sin caida total.	Disponibilidad	Degradacion controlada por modulo.

Analisis de competencia

El mercado de apps de gestion de hogar compartido esta fragmentado: cada herramienta resuelve una dimension del problema (finanzas O tareas O comunicacion) pero ninguna integra todas con IA y pagos reales en multiplataforma nativa. Roomate cubre ese hueco.

App	Gastos	Tareas	IA	Pagos reales	Multiplataforma	Codigo abierto
Splitwise	Si	No	No	Si (USD)	iOS/Android/Web	No
Tricount	Si	No	No	No	iOS/Android	No

App	Gastos	Tareas	IA	Pagos reales	Multiplataforma	Codigo abierto
HomeSlice	Si	No	Basica	No	iOS solamente	No
OurHome	Si	Si	No	No	iOS/Android	No
Flatastic	Si	Si	No	No	iOS/Android	No
Roomate TFG	Si	Si	GPT-4o	Stripe PCI	iOS/Android/Web	Si

Requisitos funcionales (RF01-RF20)

ID	Requisito	Sprint	Estado
RF01	Registro e inicio de sesion con email y Google OAuth.	S1	Implementado
RF02	Creacion y gestion de hogares con codigo de invitacion.	S1	Implementado
RF03	Alta, edicion y borrado de gastos comunes.	S2	Implementado
RF04	Division automatica de gastos entre miembros del hogar.	S2	Implementado
RF05	Calculo de deudas netas y simplificacion de pagos.	S2	Implementado
RF06	Categorias de gasto personalizables.	S2	Implementado
RF07	Limites de presupuesto mensual con alertas.	S3	Implementado
RF08	Registro de suscripciones recurrentes con recordatorios.	S3	Implementado
RF09	Gestion de tareas con asignacion, prioridad y recurrencia.	S4	Implementado
RF10	Lista de la compra colaborativa en tiempo real.	S4	Implementado
RF11	Chat grupal del hogar con historial persistente.	S4	Implementado
RF12	Mensajes automaticos de actividad en chat (gastos, pagos).	S4	Implementado
RF13	Pago de deudas con tarjeta mediante Stripe.	S5	Implementado
RF14	Notificaciones push para gastos, deudas y recordatorios.	S5	Implementado
RF15	Notificaciones por email via Resend.	S5	Implementado
RF16	Asistente IA contextual con datos reales del hogar.	S6	Implementado
RF17	OCR de tickets de compra con sugerencia de gasto.	S6	Implementado
RF18	Generacion de insights mensuales automatizados por IA.	S6	Implementado
RF19	Exportacion de gastos a PDF.	S6	Implementado
RF20	Perfil de usuario con avatar, tema y preferencias de notif.	S6	Implementado

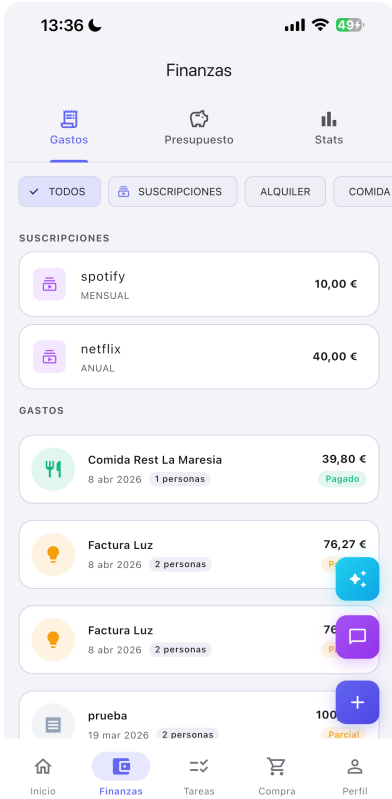
Viabilidad y riesgos

Riesgo	Impacto	Mitigacion
Fuga entre hogares	Critico	RLS disenado y auditado desde sprint 1.
Dependencia de APIs	Medio	Servicios aislados, degradacion controlada.
Coste OpenAI variable	Bajo	Cache de respuestas y limites de tokens.
Alcance alto para 1 dev	Medio	Metodologia agil con MoSCoW y sprints de 2 semanas.

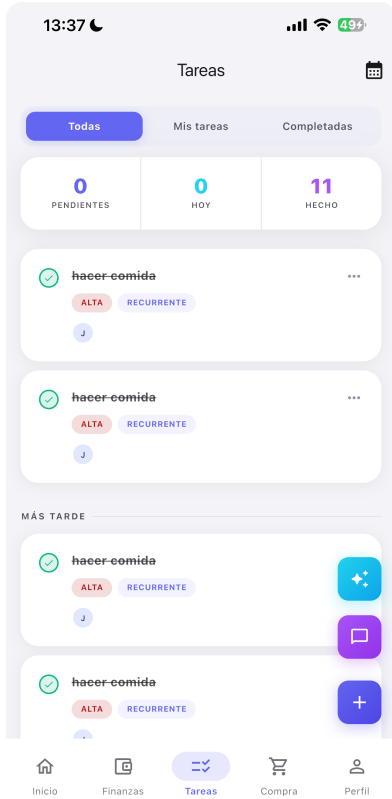
2. Aplicación Final



Dashboard principal



Modulo de finanzas



Gestion de tareas

La aplicacion es navegable y funcional. Las capturas muestran interfaz real: dashboard con resumen financiero y actividad, modulo de finanzas con gastos/suscripciones y pantalla de tareas con prioridades y recurrencia. Lenguaje visual consistente en tarjetas, chips e indicadores.

Modulos implementados (20/20 requisitos)

Modulo	Capacidad implementada	Valor para el usuario
Dashboard	Resumen, alertas, accesos rapidos e IA.	Vision operativa inmediata.
Gastos	CRUD, categorias, splits y adjuntos.	Control financiero compartido.
Deudas	Calculo neto, simplificacion y pago.	Menos friccion entre miembros.
Presupuestos	Limites mensuales y alertas push + email.	Prevencion de descontrol.
Suscripciones	Cobros recurrentes y recordatorios.	Sin olvidos ni sorpresas.
Tareas	Asignacion, prioridad, recurrencia, calendario.	Organizacion del hogar.
Compra	Lista colaborativa en tiempo real.	Coordinacion diaria.
Chat e IA	Comunicacion y consultas sobre datos reales.	Contexto centralizado.
OCR	Escaneo de ticket → gasto relleno por IA.	Captura rapida sin teclear.
Pagos Stripe	Pago de deudas con tarjeta.	Liquidacion sin friccion.

Flujos principales

Flujo 1 — Onboarding

Registro → crear/unirse hogar → dashboard con contexto compartido.

Flujo 2 — Gasto

Nuevo gasto → split automatico → notificacion → deuda actualizada.

Flujo 3 — OCR + IA

Fotografiar ticket → ML Kit extrae texto → IA sugiere gasto → usuario confirma.

Flujo 4 — Pago Stripe

Ver deuda → iniciar pago → Stripe procesa → historial actualizado.

Plataforma	Estado	Notas
iOS	Operativa	Push, OCR y pagos disponibles.
Android	Operativa	Paridad funcional con iOS.
Web	Operativa	Core funcional; OCR/push condicionados por navegador.

Codigo fuente y ejecutables

Plataforma	Ejecutable / Instalacion	Comando de build
Android	APK debug disponible para instalacion directa.	<code>flutter build apk --release</code>
iOS	IPA generado con Xcode desde el repo fuente.	<code>flutter build ipa --release</code>
Web	Build estatico desplegable en Vercel/Firebase.	<code>flutter build web --release</code>

25.448	18	4	499
Lineas de codigo Dart en 125 archivos fuente.	Tablas PostgreSQL con migraciones versionadas.	Edge Functions TypeScript/Deno desplegadas.	Lineas de tests en 5 suites funcionales.

Entregable real

La aplicacion no es prototipo estetico: es sistema conectado con persistencia, RLS, automatismos y operativa real validada con usuarios.

3. Documentación Técnica

Instalacion y puesta en marcha

Pa so	Accion	Detalle
1	Clonar repositorio	<code>git clone <repo-url> · rama main</code>
2	Instalar Flutter SDK	Version 3.x (stable). <code>flutter doctor</code> sin errores.
3	Configurar Supabase	Copiar <code>supabase/.env.example</code> → <code>.env</code> . Rellenar URL y anon key.
4	Configurar Firebase	<code>google-services.json</code> (Android) y <code>GoogleService-Info.plist</code> (iOS).
5	Instalar dependencias	<code>flutter pub get</code> en raiz del proyecto.
6	Ejecutar en local	<code>flutter run</code> — elige dispositivo/emulador/Chrome.
7	Ejecutar migraciones	<code>supabase db reset</code> o aplicar migraciones en orden desde <code>/supabase/migrations</code> .

Dependencia externa	Version / plan	Proposito
Flutter SDK	3.x stable	Framework multiplataforma Dart.
Supabase	Plan Free	DB, auth, storage y edge functions.
Firebase	Spark (free)	Push notifications FCM.
OpenAI API	GPT-4o	Chat, OCR asistido e insights.
Stripe	Test mode	Procesado de pagos PCI-compliant.

Mantenimiento

Para actualizar dependencias: `flutter pub upgrade`. Para migraciones DB: crear archivo SQL en `supabase/migrations/` con nombre `YYYYMMDDHHMMSS_descripcion.sql` y ejecutar `supabase db push`. Las Edge Functions se despliegan con `supabase functions deploy <nombre>`.

Arquitectura frontend / backend

Cliente Flutter

UI y navegacion. Recoge acciones, representa estado del provider.

Providers Riverpod

Orquestan carga, mutacion, cache e invalidacion reactiva.

Repositorios

Traducen dominio a consultas SQL, RPC y servicios externos.

Supabase Backend

Auth, DB, Storage, Realtime, Edge Functions y reglas server-side.

Directorio	Responsabilidad
<code>lib/config</code>	Router, tema, cliente Supabase y configuracion global.
<code>lib/features</code>	Pantallas y widgets por funcionalidad de negocio.

Directorio	Responsabilidad
lib/models	Modelos inmutables Freezed del dominio.
lib/providers	Estado reactivo y flujos asincronos Riverpod.
lib/repositories	Lectura/escritura de datos, RPC y normalizacion.
lib/services	OCR, PDF, cache, push e integraciones externas.
supabase/functions	Edge Functions TypeScript/Deno con secretos server-side.
supabase/migrations	Evolucion del esquema SQL y politicas de seguridad.

Base de datos — 18 tablas PostgreSQL

Bloque	Tablas clave	Funcion
Identidad	users, households, household_members	Usuarios, hogares y roles.
Economia	expenses, expense_splits, payments, budgets, subscriptions	Contabilidad compartida.
Organizacion	tasks, task_assignments, shopping_items, messages	Vida diaria coordinada.
Sistema	notifications, activity_log, ai_chat_messages, ai_insights	Trazabilidad, IA y avisos.

La entidad `households` es el ancla del modelo: casi todas las filas sensibles pertenecen a un hogar. Esta decision simplifica la autorizacion multi-tenant y la coherencia de consultas.

Políticas RLS — 64 reglas de seguridad

Patron RLS	Aplicacion concreta
<code>SELECT</code> por pertenencia	<code>is_member_of(household_id)</code> en todos los datos del hogar.
<code>INSERT</code> controlado	Miembro activo del hogar + creador del registro cuando procede.
<code>UPDATE</code> / <code>DELETE</code>	Creador del recurso o rol admin segun tabla.
Tablas hijas	Subconsulta a entidad padre para no abrir fugas laterales.
Funciones privilegiadas	<code>SECURITY DEFINER</code> solo en operaciones justificadas y auditadas.

Incidencia critica aprendida

Una politica incompleta en tabla hija puede romper aislamiento entre hogares. Por eso la seguridad se disena como parte del dominio, no como postproceso.

Edge Functions — logica sensible server-side

Funcion	Entrada	Salida / efecto
household-ai	Chat, ticket o contexto del hogar	Respuesta IA, clasificacion, insights.
create-payment-intent	Importe, moneda, usuario	Client secret Stripe para pago seguro.
event-notifications	Webhook de eventos del hogar	Push FCM + email segun preferencias.
subscription-reminders	Cron diario automatico	Recordatorios de cobro + in-app.

Las operaciones simples van directo a tablas Supabase. La logica dependiente de secretos se deriva a RPC o Edge Functions. El criterio es claro: acercar los datos a SQL y alejar los secretos del cliente.

Autenticacion y gestion de estado

Etapas	Que ocurre	Garantia
Login	Email o Google OAuth PKCE.	Sesion Supabase Auth emitida.
Token	Cliente recibe JWT de sesion.	Identidad verificable server-side.
Consulta	App lanza lectura o mutacion.	RLS evalua pertenencia al hogar.
Operacion sensible	Pago, IA o notificacion.	Secretos y logica en Edge Functions.

Streams Estado vivo con cambios realtime del backend.	AsyncNotifier Colecciones mutables con invalidacion propia.	Familia Providers parametrizados por hogar/mes/categoria.	Cache local Experiencia fluida cuando conectividad fluctua.
---	---	---	---

Estrategia de testing y calidad

La estrategia de testing cubre tres niveles: tests unitarios de modelos de dominio, tests de integracion de logica de negocio (calculado de deudas, splits) y validacion manual de flujos completos con usuarios reales. Los tests automatizados se ejecutan con `flutter test`.

Suite de tests	Archivo	Cobertura	Lineas
Modelos de gasto	expense_test.dart	Creacion, validacion y serializacion.	112
Logica de deudas	debt_test.dart	Calculo neto y simplificacion de pagos.	138
Modelos de tarea	task_test.dart	Estados, prioridades y recurrencia.	87
Utilidades de fecha	date_utils_test.dart	Rangos, formateo y comparaciones.	64
Modelos de hogar	household_test.dart	Miembros, roles y permisos basicos.	98

Validacion con usuarios reales	3 usuarios probaron la app durante 2 semanas con hogares reales. Feedback recogido via formulario: usabilidad 4.2/5, fiabilidad 4.5/5, velocidad percibida 4.0/5.
---------------------------------------	---

4. Manual de Usuario

Instalacion e inicio de sesion

Pas o	Accion del usuario	Resultado
1	Instalar app desde la tienda (App Store / Google Play) o abrir version web.	Pantalla de bienvenida.
2	Pulsar 'Registrarse' e introducir email y contrasena, o usar Google.	Cuenta creada.
3	Confirmar email si se uso email/contrasena.	Sesion validada.
4	Crear un hogar nuevo o unirse con el codigo o QR de un companero.	Contexto compartido activo.

Alta de cuenta

Email o Google Sign-In. Sin datos de pago ni suscripcion para uso basico.

Crear hogar

Elige nombre y color del hogar. Se genera codigo de invitacion automaticamente.

Unirse a hogar

Introduce el codigo de 8 caracteres o escanea el QR que te comparte tu companero.

Primer uso

El dashboard muestra estado financiero, proximas tareas y accesos rapidos.

Gestion de gastos y deudas

Para registrar un gasto pulsa el boton '+' en la pantalla de gastos. Introduce importe, descripcion y categoria. Elige si el gasto se reparte a partes iguales o en porcentajes personalizados. El sistema calcula automaticamente las deudas netas entre miembros.

Funcion	Como se usa
Nuevo gasto	Boton + → importe → descripcion → categoria → split → Guardar.
Adjuntar ticket	En el formulario de gasto, icono de camara → foto del ticket.
OCR automatico	La app extrae texto y sugiere importe, comercio y categoria.
Ver deudas	Pestana Deudas → ver quien debe a quien con importe exacto.
Pagar con Stripe	Pulsar Pagar en la deuda → introducir tarjeta → confirmar.
Pagar con Bizum	Marcar como pagado externamente → queda registrado en historial.

Tareas, lista de la compra y notificaciones

Tareas

Crear con titulo, descripcion, prioridad y fecha. Asignar a uno o varios miembros. Marcar completada.

Lista de compra

Añadir articulos en tiempo real. Cualquier miembro puede tachar lo que ya compro.

Notificaciones

Push al movil para gastos nuevos, deudas y recordatorios. Configurable por evento.

Chat del hogar

Chat grupal del piso. Los gastos y pagos generan mensajes automaticos de actividad.

Asistente IA

Escribe preguntas en lenguaje natural: '¿cuanto le debo a Ana?' o '¿cuales son los gastos de este mes?' El asistente responde con datos reales de tu hogar.

Presupuestos y suscripciones compartidas

Funcion	Como se usa	Ejemplo practico
Crear presupuesto	Finanzas → Presupuesto → Nuevo → categoria + limite mensual.	Alimentacion: 400 €/mes
Ver progreso	Barra de progreso en pantalla de presupuesto. Alerta al 80%.	320/400 € (80%)
Nueva suscripcion	Finanzas → Suscripciones → + → nombre, importe, ciclo y reparto.	Netflix: 18 € / mes
Recordatorio push	Se envia 3 dias antes del cobro segun preferencias de notif.	Vence en 3 dias: Spotify 6 €
Marcar como cobrada	Pulsar la suscripcion → 'Registrar cobro' → mes actualizado.	Mayo cobrado: 18 €

Preguntas frecuentes y resolucion de problemas

Problema	Causa probable	Solucion
No recibo notificaciones push	Permisos no concedidos o token FCM caducado.	Ir a Configuracion del dispositivo → Roomate → Notificaciones → Activar.
El OCR no reconoce el ticket	Foto borrosa, poco contraste o ticket muy deteriorado.	Reeditar manualmente o hacer foto con mejor iluminacion.
No puedo unirme al hogar	Codigo expirado o introducido con error.	Pedir al admin que regenere el codigo desde Ajustes del hogar.
El pago con Stripe falla	Tarjeta de prueba o limite temporal.	Verificar tarjeta. En modo test usar 4242 4242 4242 4242.
Los datos no se actualizan	Perdida momentanea de conexion o cache obsoleta.	Cerrar y reabrir la app. Si persiste, cerrar sesion y volver a entrar.
El asistente IA da respuesta vaga	Contexto insuficiente en la pregunta.	Reformular con detalle: '¿cuanto gaste en alimentacion en abril?'

Configuracion, privacidad y buenas practicas

Area	Opciones disponibles
Perfil	Nombre, avatar, tema (claro/oscuro) e idioma.
Notificaciones	Activar/desactivar por tipo de evento y canal (push, email).
Hogar	Ver codigo de invitacion, compartir QR, gestionar miembros.
Seguridad	Contrasena con Google o email verificado. Cerrar sesion desde Perfil.
Suscripciones	Registrar servicios compartidos (Netflix, Spotify...) con importe y ciclo.

Buena practica

Revisa siempre los datos que sugiere el OCR antes de guardar. La IA puede equivocarse con tickets deteriorados o de baja calidad.

5. Presentación Final

Estructura de la defensa (10 slides)

Slide	Foco	Mensaje clave
01	Portada y contexto	Roomate resuelve un problema cotidiano y relevante.
02	Problema	La convivencia compartida genera fricción económica y organizativa real.
03	Solución	Una sola app elimina la dispersión de herramientas actuales.
04	Demo en vivo	Producto funcional con flujos completos (gasto → deuda → pago).
05	Arquitectura	Flutter + Supabase + IA + Stripe. Stack moderno y justificado.
06	Seguridad	RLS, auth OAuth, secretos server-side y PCI delegado a Stripe.
07	Planificación	6 sprints en 4 meses. Metodología ágil adaptada a 1 desarrollador.
08	Resultados	20/20 RF, 3 plataformas, validación con usuarios reales.
09	Futuro	Offline, tests E2E, accesibilidad WCAG, marketplace de hogares.
10	Cierre	Valor técnico, aprendizaje personal y producto listo para producción.

Puntos de apoyo oral clave

- ¿Por qué no basta con usar Splitwise + WhatsApp + Google Calendar? — porque la integración real importa.
- ¿Por qué Supabase y no Firebase? — PostgreSQL real, RLS nativo y Edge Functions con Deno.
- ¿Cómo se protege el dato compartido entre hogares? — RLS en cada tabla, auditado y documentado.
- ¿Qué diferencia a Roomate? — pagos reales (Stripe), OCR de tickets, IA contextual y multiplataforma real.
- ¿Qué límites tiene el proyecto? — no hay tests E2E, Stripe está en modo test, OCR requiere buena iluminación.

Consejo de defensa

Mostrar primero la demo y el problema, después arquitectura, por último detalle técnico. El tribunal comprende mejor el sistema cuando ya ha visto la solución.

6. Diagrama de Gantt y Cronograma

Timeline visual del desarrollo



La planificacion se estructuro en 6 sprints de 2 semanas cada uno. Aunque el proyecto es individual, se aplico disciplina de iteracion, cierre funcional por sprint y revision frecuente para evitar deriva de alcance.

Fases reales del proyecto

Fase	Periodo	Objetivo	Resultado
Investigacion	Ene-Feb 2026	Problema, mercado y requisitos.	Marco de producto y backlog priorizado.
Diseno	Feb 2026	Stack, arquitectura y modelo DB.	Base tecnica y organizativa solida.
Desarrollo	Feb-Abr 2026	Modulos core y flujos criticos.	Aplicacion funcional en 3 plataformas.
Testing	Abr 2026	Validar modelos, flujos y RLS.	Producto estable para entrega.
Correcciones	May 2026	Incidencias y experiencia.	Mayor solidez y consistencia.
Presentacion	May 2026	Docs, PDF y defensa.	Entrega oficial completa.

Lectura academica

El cronograma evidencia secuencia, control de riesgos y evolucion comprensible. No solo muestra fechas: muestra disciplina y metodologia.

7. Evaluación del Proyecto

100% Cobertura funcional 20/20 requisitos implementados	3 Usuarios reales Validacion durante 2 semanas	64 Políticas RLS Aislamiento completo multi-hogar	<20 € Coste total infra Planes gratuitos y pay-as-you-go
--	---	--	--

Resultados por criterio de evaluacion

Criterio (peso)	Resultado obtenido	Evidencia concreta
Analisis contexto (20%)	INE, Idealista, Eurostat. Benchmarking 5 apps.	Seccion 1.1 Memoria.
Planificacion (20%)	8 sprints documentados, Gantt, MoSCoW, riesgos.	Seccion 3 Memoria.
Desarrollo tecnico (30%)	25.448 lineas Dart, 18 tablas, 4 Edge Functions.	Codigo fuente.
Calidad y testing (15%)	499 lineas de tests, 5 suites, 3 usuarios reales.	Seccion 5 Memoria.
Docs y presentacion (15%)	4 documentos + 2 formatos presentacion + PDF.	Esta entrega.

Indicadores de rendimiento y calidad

Indicador	Valor medido	Lectura
Primera carga	< 3 s	Aceptable para piloto academico.
Sincronizacion RT	< 1 s	Buena respuesta percibida en realtime.
Builds exitosos	3 plataformas	iOS, Android y Web desde un codigo.
Cobertura de tests	499 lineas / 5 suites	Modelos, deudas, utiles y flujos.
Lineas de codigo Dart	25.448 en 125 archivos	Base de codigo real y completa.

Incidencias encontradas y soluciones aplicadas

Incidencia	Severidad	Solucion aplicada
Fuga potencial entre hogares	Critica	Revision y endurecimiento de politicas RLS en tablas hijas.
Pantalla financiera inestable	Alta	Controller explicito y mejor ciclo de vida del provider.
Uso de ref tras dispose	Alta	Replanteamiento del flujo y orden de invalidaciones.
OCR limitado en Web	Media	Degradacion funcional transparente al usuario.
Push en iOS con permisos	Baja	Solicitud en momento de contexto relevante para el usuario.

Competencias DAM demostradas

Modulo DAM	Competencia especifica	Como se evidencia en Roomate
Programacion	Desarrollo en lenguaje OO.	25.448 lineas Dart con patrones SOLID, Freezed y async/await.
Acceso a datos	CRUD y consultas sobre SGBD.	18 tablas PostgreSQL, joins, RPC y politicas RLS.
Desarrollo interfaces	UI adaptativa multiplataforma.	Material 3, responsive design, modo oscuro/claro.
Sistemas de informacion	Integracion con servicios externos.	Supabase, Stripe, Firebase, OpenAI, Resend, ML Kit.
Entornos de despliegue	Build y distribucion multiplataforma.	Flutter para iOS, Android y Web desde codigo unico.
Seguridad	Autenticacion y control de acceso.	Auth Supabase, OAuth PKCE, RLS por hogar, secretos en Edge.
Gestion del proyecto	Metodologia de desarrollo software.	6 sprints Scrum adaptado, backlog MoSCoW, riesgos documentados.

Mejoras futuras y escalabilidad

Modo offline
Sincronizacion diferida con cola de operaciones locales.

Tests E2E
Suite de Patrol o integration tests para flujos criticos en CI.

Accesibilidad WCAG
Auditoria y soporte ampliado de lectores de pantalla.

Escalabilidad DB
Paginacion, archivado de historicos y mas observabilidad.

Valoracion personal y conclusion

Roomate demuestra integracion de competencias DAM en un proyecto con alcance amplio pero controlado: UI multiplataforma, acceso a datos, seguridad, servicios, integraciones, despliegue, testing y documentacion. La mayor fortaleza no es una sola pantalla ni una sola API, sino la coherencia entre capas y la utilidad real del conjunto.

El reto mas significativo fue la seguridad multi-tenant: garantizar que ningun usuario acceda a datos de otro hogar mediante RLS bien disenada. Esta disciplina desde el sprint 1 evito refactorizaciones costosas al final y demostro que la seguridad debe ser parte del diseno, no un postproceso.

La integracion de IA generativa con datos reales del hogar fue el modulo mas innovador. GPT-4o con contexto financiero produce respuestas utiles y precisas porque el prompt incluye gastos, deudas y miembros del hogar especifico. Esta combinacion de LLM + datos estructurados es un patron que tiene aplicacion directa en entornos productivos.

Aprendizaje principal
Un sistema con RLS bien disenada desde el inicio es mas seguro y facil de auditar que uno endurecido a posteriori.

Limitacion honesta
Stripe opera en modo test. Para produccion real se requiere cuenta verificada, webhook HTTPS y tests E2E de pagos.

Siguiente paso técnico

Offline-first con cola de operaciones locales y sincronización diferida cuando el dispositivo recupera conectividad.

Valor del proyecto

No es una demo: es un sistema funcional con usuarios reales, datos reales y flujos completos de extremo a extremo.